



# What Is Your Algorithmic Core?

EGOR SUVOROV



20  
26



# What is your algorithmic core?

Engineering

Bloomberg

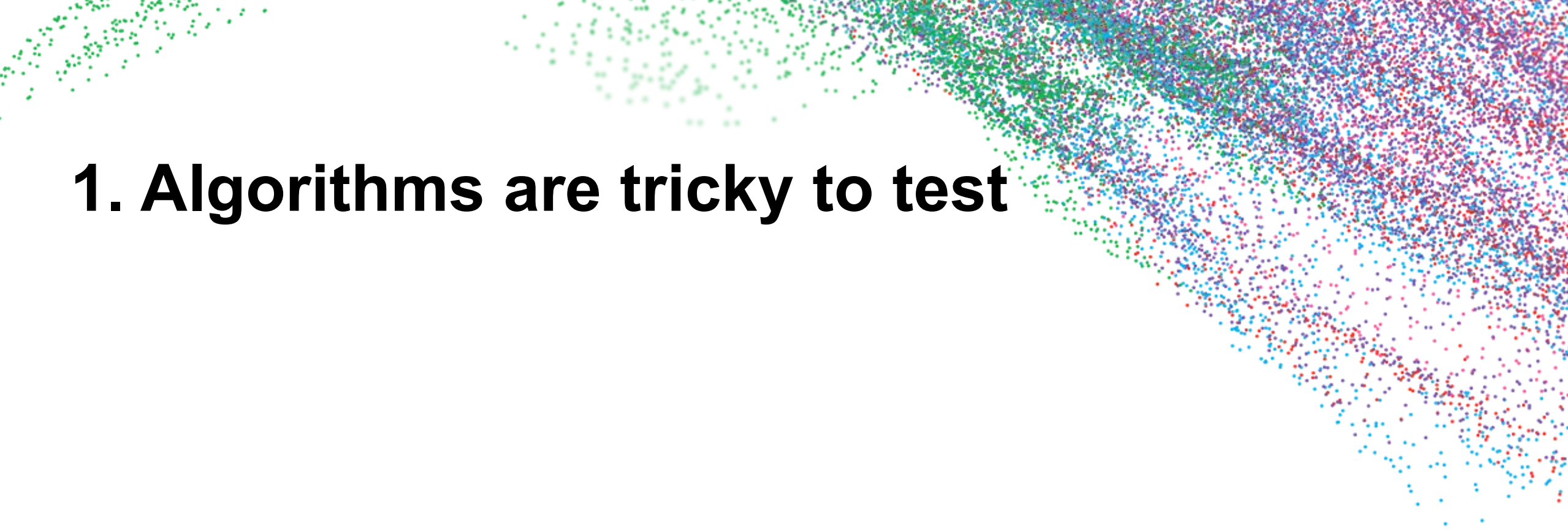
CppCon 2026  
September 17, 2026

Egor Suvorov  
Senior Software Engineer

[TechAtBloomberg.com](https://TechAtBloomberg.com)

# Plan

1. Algorithms are tricky to test
2. Algorithms are stubborn
3. Bugs hide until they don't
4. Extracting algorithms
5. Signs of an algorithm



# 1. Algorithms are tricky to test

# How do we gain confidence in code?

**TechAtBloomberg.com**

© 2026 Bloomberg Finance L.P. All rights reserved.

**Bloomberg**

Engineering

# How do we gain confidence in code?

- **Design:** library reuse, types, API fool-proofness, invariants

# How do we gain confidence in code?

- **Design:** library reuse, types, API fool-proofness, invariants
- **Reading:** code review, static analysis, formal verification

# How do we gain confidence in code?

- **Design:** library reuse, types, API fool-proofness, invariants
- **Reading:** code review, static analysis, formal verification
- **Running:** test cases, fuzzing, replays, sanitizers

# How do we gain confidence in code?

- **Design:** library reuse, types, API fool-proofness, invariants
- **Reading:** code review, static analysis, formal verification
- **Running:** test cases, fuzzing, replays, sanitizers
- **Extra fixtures:** mock FS, network lags, thread reordering

# How do we gain confidence in code?

- **Design:** library reuse, types, API fool-proofness, invariants
- **Reading:** code review, static analysis, formal verification
- **Running:** test cases, fuzzing, replays, sanitizers
- **Extra fixtures:** mock FS, network lags, thread reordering
- **SDLC:** QA, gradual rollouts, backups...

# How do we gain confidence in code?

- **Design:** library reuse, types, API fool-proofness, **invariants**
- **Reading:** **code review**, static analysis, **formal verification**
- **Running:** **test cases**, fuzzing, replays, sanitizers
- **Extra fixtures:** mock FS, network lags, thread reordering
- **SDLC:** QA, gradual rollouts, backups...

We need a good set of tests anyway, assume 100% branch coverage

# Engineering bugs: Lifetime

```
void onClick() {  
    string url = "https://example.com/?q=" + textBox.text(); // Save the URL  
    httpClient.connect([&]() { // callback  
        httpClient.send(url, [&]() { /* callback */ });  
    });  
}
```

# Engineering bugs: Lifetime

```
void onClick() {  
    string url = "https://example.com/?q=" + textBox.text(); // Save the URL  
    httpClient.connect([&]() { // callback  
        httpClient.send(url, [&]() { /* callback */ });  
    });  
}
```

- The `url` local variable is captured in a lambda by reference
- Lambda is invoked way after `onClick` completes
- Use-after-free
- Likely caught by line coverage

# Algorithm: Substring matching

Prompt: “Create a `matchSubstring(haystack, needle)` function in C++ that works in linear time”.

**Bloomberg**

Engineering

# Algorithm: Substring matching

Prompt: “Create a `matchSubstring(haystack, needle)` function in C++ that works in linear time”. LLM uses the KMP algorithm.

```
// Build LPS (Longest Prefix Suffix) array
vector<int> lps(m, 0);
int len = 0, i = 1;
while (i < m) {
    if (pattern[i] == pattern[len]) lps[i++] = ++len;
    else if (len != 0) len = lps[len - 1];
    else lps[i++] = 0;
}
```

**Bloomberg**

Engineering

# Asking LLM to add tests: Got 17 black-box cases

```
assert(matchSubstring("", "") == 0); // Empty
assert(matchSubstring("abc", "abc") == 0); // Exact
// ... checks matches at beginning, middle, end, nowhere...
// Pattern appears later after partial mismatches
assert(matchSubstring("aaaaab", "aab") == 2);
assert(matchSubstring("mississippi", "issip") == 4);
assert(matchSubstring("aaacaaaaac", "aaaac") == 4);
// Common textbook-style example
assert(matchSubstring("abxabcabcaby", "abcaby") == 6);
```

**Bloomberg**

Engineering

# Are tests really good?

- We had full line and branch coverage

```
// Build LPS (Longest Prefix Suffix) array
vector<int> lps(m, 0);
int len = 0, i = 1;
while (i < m) {
    if (pattern[i] == pattern[len]) lps[i++] = ++len;
    else if (len != 0) len = lps[len - 1];
    else lps[i++] = 0;
}
```

**Bloomberg**

Engineering

# Are tests really good?

- We had full line and branch coverage
- Removing one line ...

```
// Build LPS (Longest Prefix Suffix) array
vector<int> lps(m, 0);
int len = 0, i = 1;
while (i < m) {
    if (pattern[i] == pattern[len]) lps[i++] = ++len;
    else if (len != 0) len = lps[len - 1];
    else lps[i++] = 0;
}
```

# Are tests really good?

- We had full line and branch coverage
- Removing one line does not make tests fail => not great
- LLM knew about the algorithm it used

```
// Build LPS (Longest Prefix Suffix) array
vector<int> lps(m, 0);
int len = 0, i = 1;
while (i < m) {
    if (pattern[i] == pattern[len]) lps[i++] = ++len;
    else if (len != 0) len = lps[len - 1];
    else lps[i++] = 0;
}
```

**Bloomberg**

Engineering

# Mutation testing

- Requirement: changing a correct program a bit should either:
  - Keep the program correct
  - Make the program incorrect **and** fail existing tests
- Protects from small bugs and typos, even when done manually
  - Pins down most corner cases
- <https://sqlite.org/testing.html> does that automatically:
  - “In other words, we want to show not only that every branch instruction both jumps and falls through but also that every branch is doing useful work and that the test suite is able to detect and verify that work.”

# Improving tests

**TechAtBloomberg.com**

© 2026 Bloomberg Finance L.P. All rights reserved.

**Bloomberg**

Engineering

# Improving tests

- Random strings:  $[a-e]\{0, 50\}$ , 10'000 tests pass

# Improving tests

- Random strings:  $[a-e]\{0, 50\}$ , 10'000 tests pass
- Coverage-based libfuzzer: Dozens of millions of runs, all pass
  - All branches were already covered!

# Improving tests

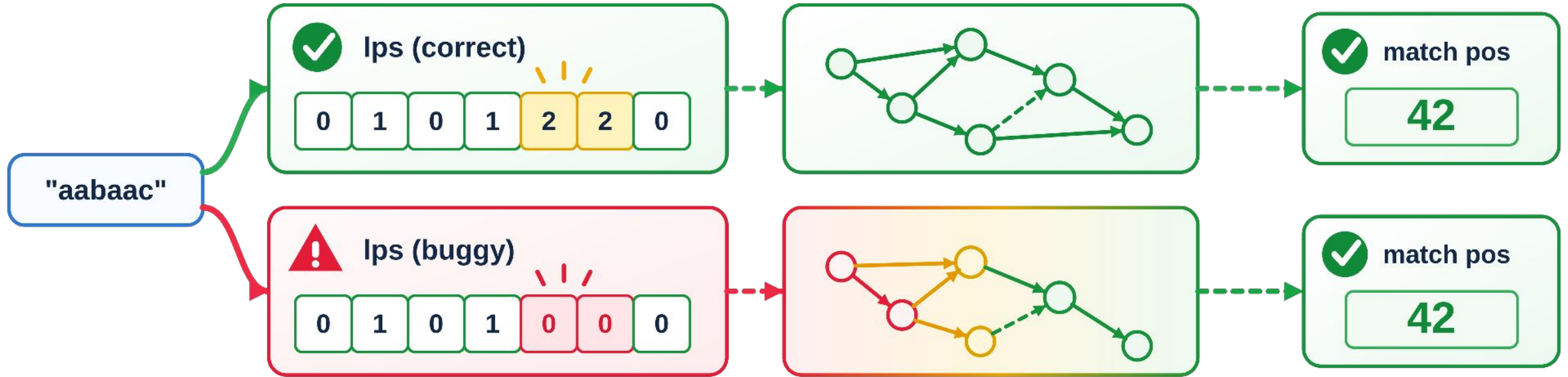
- Random strings: `[a-e]{0, 50}`, 10'000 tests pass
- Coverage-based libfuzzer: Dozens of millions of runs, all pass
  - All branches were already covered!
- Root cause: Input corner cases  $\neq$  algorithm's corner cases

```
assert (matchSubstring("abababacaba", "abacaba") == 2) ;
```

# Random test generators

- What is the probability space?
- What answers are likely?
  - Random string is not a substring of another random string
- Any structure induced/avoided by the generator?
  - “aaaaaaaaaab” string is unlikely
  - Multiple matches
  - Repeated substrings
  - Almost matches
- There are known “beautiful” domain-specific tests

# White-box testing is better for algorithms



```
assert (buildLPS("aabaac") == vector<int>(
    {0, 1, 0, 1, 2, 2, 0}));
```

- If the algorithm changes, all corner cases change too
- LLM can write tests

# STL Design Principle: Law of Useful Return

*“If you’ve already done the work to get some useful result, don’t throw it away. Return it to the caller...” — Alexander Stepanov, Daniel Rose*

| Bad                                   | Better                                                                               |
|---------------------------------------|--------------------------------------------------------------------------------------|
| <code>bool binary_search(...)</code>  | <code>int lower_bound(...)</code>                                                    |
| <code>bool matchSubstring(...)</code> | <code>vector&lt;int&gt; buildLSP(...)</code><br><code>int matchSubstring(...)</code> |
| <code>void magicSort(...)</code>      | <code>int getSortKey(...)</code>                                                     |

# Add self-checks: You might already have tests

- Assert preconditions, internal invariants, postconditions
  - What would be your debug output?
  - Before binary search: All elements are sorted
  - During binary search: All to the left  $\leq$  current  $\leq$  all to the right
  - After sorting: All pairs (not just adjacent) elements are sorted
- An independent naive implementation gives the same answer
- Vary magic constants (0...1000)
  - IntroSort is three algorithms: insertion sort + heap sort + quick sort
  - I/O buffer: Does your concatenation work?
  - Block size for MyDeque — small tests will cover more cases

# Takeaway: Two kinds of bugs

**TechAtBloomberg.com**

© 2026 Bloomberg Finance L.P. All rights reserved.

**Bloomberg**

Engineering

# Takeaway: Two kinds of bugs

1. Engineering: Language-specific
  - a. Likely caught by smoke tests or line coverage
  - b. Might be hard to reproduce (bad\_alloc, network lags, sad path, invalid API calls)

# Takeaway: Two kinds of bugs

## 1. Engineering: Language-specific

- a. Likely caught by smoke tests or line coverage
- b. Might be hard to reproduce (bad\_alloc, network lags, sad path, invalid API calls)

## 2. Algorithmic: Language-agnostic

- a. Easily survive all usual testing, but easy to reproduce
- b. *“Beware of bugs ...; I have only proved the code correct, not tried it.”* - Donald Knuth
- c. You should know what algorithm you use for good tests

# Takeaway: Two kinds of bugs

## 1. Engineering: Language-specific

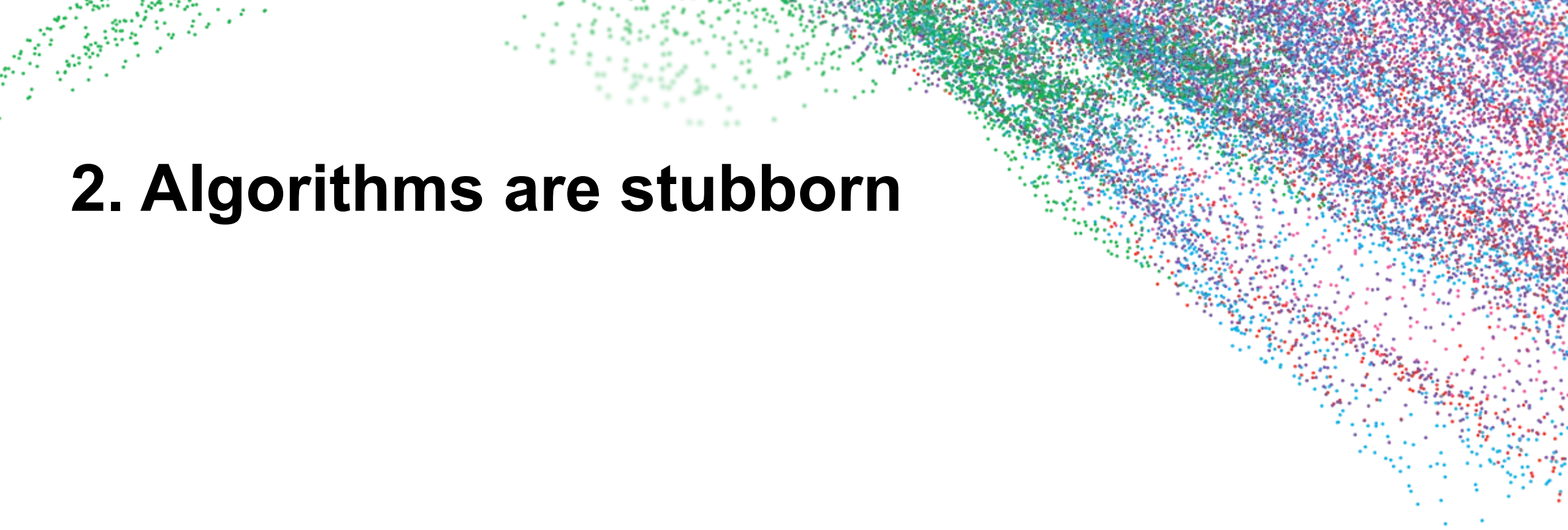
- a. Likely caught by smoke tests or line coverage
- b. Might be hard to reproduce (bad\_alloc, network lags, sad path, invalid API calls)

## 2. Algorithmic: Language-agnostic

- a. Easily survive all usual testing, but easy to reproduce
- b. *“Beware of bugs ...; I have only proved the code correct, not tried it.”* - Donald Knuth
- c. You should know what algorithm you use for good tests

## 3. Mixed

- a. Concurrency, lock-free algorithms: Need language support
- b. Exception safety: Extra complexity induced by C++
- c. Lots of corner cases in the problem statement



## 2. Algorithms are stubborn

# Print packages hierarchy

```
class A { B b; C c; }; // A.h, level=4
class B {           }; // B.h, level=1
class C { D d; }; // C.h, level=3
class D { E e; }; // D.h, level=2
class E {           }; // E.h, level=1
```

Expected output: by level, then alphabetically

A.h C.h D.h B.h E.h

# Calculate levels (recursion with memoization)

```
int calcLevels(File* f) {  
    int &lev = levels[f];  
    if (lev) return lev;  
    lev = 1;  
    for (auto c : f->classes)  
        for (auto m : c->members)  
            for (auto c : m->type().classes())  
                lev = max(lev, calcLevels(class2File[c]) + 1);  
    return lev;  
}
```



# Calculate levels (recursion with memoization)

```
int calcLevels(File* f) {  
    int &lev = levels[f];  
    if (lev) return lev;  
    lev = 1;  
    for (auto c : f->classes)  
        for (auto m : c->members)  
            for (auto c : m->type().classes())  
                lev = max(lev, calcLevels(class2File[c]) + 1);  
    return lev;  
}
```



# Print packages hierarchy

```
class A { B b; C c; }; // A.h, level=4
class B {           }; // B.h, level=1
class C { D d; };    // C.h, level=3
class D { E e; };    // D.h, level=2
class E {           }; // E.h, level=1
```

Expected output: by level, then alphabetically

A.h C.h D.h B.h E.h

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=  
class B {           }; // B.h, level=  
class C { D d; }; // C.h, level=  
class D { E e; }; // D.h, level=  
class E { C *c; }; // E.h, level=
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=1
class B {                }; // B.h, level=
class C { D d; }; // C.h, level=
class D { E e; }; // D.h, level=
class E { C *c; }; // E.h, level=
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=1
class B {                }; // B.h, level=1
class C { D d; }; // C.h, level=
class D { E e; }; // D.h, level=
class E { C *c; }; // E.h, level=
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=2 (B+1)
class B {                }; // B.h, level=1
class C { D d; }; // C.h, level=
class D { E e; }; // D.h, level=
class E { C *c; }; // E.h, level=
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=2
class B {                }; // B.h, level=1
class C { D d; }; // C.h, level=1
class D { E e; }; // D.h, level=
class E { C *c; }; // E.h, level=
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=2
class B {                }; // B.h, level=1
class C { D d; }; // C.h, level=1
class D { E e; }; // D.h, level=1
class E { C *c; }; // E.h, level=
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=2
class B {           }; // B.h, level=1
class C { D d; }; // C.h, level=1
class D { E e; }; // D.h, level=1
class E { C *c; }; // E.h, level=1
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=2
class B {                }; // B.h, level=1
class C { D d; }; // C.h, level=1
class D { E e; }; // D.h, level=1
class E { C *c; }; // E.h, level=1
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=2
class B {                }; // B.h, level=1
class C { D d; }; // C.h, level=1
class D { E e; }; // D.h, level=1
class E { C *c; }; // E.h, level=2 (C+1)
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=2
class B {           }; // B.h, level=1
class C { D d; }; // C.h, level=1
class D { E e; }; // D.h, level=1
class E { C *c; }; // E.h, level=2
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=2
class B {                }; // B.h, level=1
class C { D d; }; // C.h, level=1
class D { E e; }; // D.h, level=3 (E+1)
class E { C *c; }; // E.h, level=2
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=2
class B {           }; // B.h, level=1
class C { D d; }; // C.h, level=4 (D+1)
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=5 (C+1)
class B {                }; // B.h, level=1
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```

# Unexpected input still works

```
class A { B b; C c; }; // A.h, level=5
class B {                }; // B.h, level=1
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```

Output is the same, changed slightly, more or less reasonable:

A.h C.h D.h E.h B.h



But internals are already weird:

`calcLevels(E) → (calcLevels(C) → intermediate 1) + 1`

**Bloomberg**

Engineering

## Another change may completely break output

```
class A { B b; C c; }; // A.h, level=5
class B { A *a; }; // B.h, level=2
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```

Output is reverted back, so B comes after all of A, C, D:

A.h C.h D.h B.h E.h



Internals became weirder:

`calcLevels(B) → (calcLevels(A) → intermediate 1) + 1`

**Bloomberg**

Engineering

## But a small change “fixes” everything

```
class A { C c; B b; }; // A.h, level=  
class B { A *a; }; // B.h, level=  
class C { D d; }; // C.h, level=  
class D { E e; }; // D.h, level=  
class E { C *c; }; // E.h, level=
```



## But a small change “fixes” everything

```
class A { C c; B b; }; // A.h, level=5 (C+1)
class B { A *a; }; // B.h, level=
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```



## But a small change “fixes” everything

```
class A { C c; B b; }; // A.h, level=5
class B { A *a; }; // B.h, level=1
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```



## But a small change “fixes” everything

```
class A { C c; B b; }; // A.h, level=5
class B { A *a; }; // B.h, level=1
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```



## But a small change “fixes” everything

```
class A { C c; B b; }; // A.h, level=5
class B { A *a; }; // B.h, level=6 (A+1)
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```



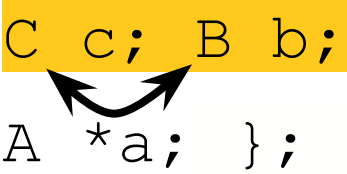
## But a small change “fixes” everything

```
class A { C c; B b; }; // A.h, level=5
class B { A *a; }; // B.h, level=6
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```



## But a small change “fixes” everything

```
class A { C c; B b; }; // A.h, level=7 (B+1)
class B { A *a; }; // B.h, level=6
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```



## But a small change “fixes” everything

```
class A { C c; B b; }; // A.h, level=7
class B { A *a; }; // B.h, level=6
class C { D d; }; // C.h, level=4
class D { E e; }; // D.h, level=3
class E { C *c; }; // E.h, level=2
```



Now B is at the front, together with A:

A.h B.h C.h D.h E.h

Internals became even weirder:

`calcLevels(B) → (calcLevels(A) → intermediate 5) + 1`

**Bloomberg**

Engineering

# What happened?

- We didn't think about cycles at all => not in contract

# What happened?

- We didn't think about cycles at all => not in contract
- Code *was* correct, but *data* changed
  - New inputs are effectively “logical UB”
  - Nobody reads the internal state => “logical UB” is invisible
  - Hence, sudden incorrect behaviour

# What happened?

- We didn't think about cycles at all => not in contract
- Code *was* correct, but *data* changed
  - New inputs are effectively “logical UB”
  - Nobody reads the internal state => “logical UB” is invisible
  - Hence, sudden incorrect behaviour
- Fixing algorithm might affect “already working” code (Hyrum's Law)

# The algorithm has a name

- Asked an LLM: “What is this algorithm?” + code
  - “It’s a memoized DFS computing longest dependency depth, used to approximate a topological sort.”
- Graph nodes ~ files
- Graph edges ~ dependencies between classes inside a file

# The algorithm has a name

- Asked an LLM: “What is this algorithm?” + code
  - “It’s a memoized DFS computing longest dependency depth, used to approximate a topological sort.”
- Graph nodes ~ files
- Graph edges ~ dependencies between classes inside a file
- The ~20 LoC algorithm hides behind boilerplate
  - We use raw pointers everywhere lifetime issues
  - (file, class) is a single identifier, but we track both pointers

# The algorithm has a name

- Asked an LLM: “What is this algorithm?” + code
  - “It’s a memoized DFS computing longest dependency depth, used to approximate a topological sort.”
- Graph nodes ~ files
- Graph edges ~ dependencies between classes inside a file
- The ~20 LoC algorithm hides behind boilerplate
  - We use raw pointers everywhere lifetime issues
  - (file, class) is a single identifier, but we track both pointers
- There are other algorithms worth testing
  - Is incomplete type a dependency?

# Extracting the algorithm with LLM (1/2)

```
template <typename Node, typename GetDeps>
class DependencyLevels {
public:
    DependencyLevels(GetDeps getDeps);
    int level(const Node& n);
    template <typename It>
    void sortByLevel(It begin, It end);
private:
    std::unordered_map<Node, int> memo;
    GetDeps getDeps;
};
```

## Extracting the algorithm with LLM (2/2)

```
auto getDeps = [&](File* f) {  
    std::vector<File*> deps;  
    for (auto c : f->classes)  
        for (auto m : c->members)  
            for (auto c2 : m->type().classes())  
                deps.push_back(class2File[c2]);  
    return deps;  
};  
DependencyLevels<File*, decltype(getDeps)> levels(getDeps);  
levels.sortByLevel(allFiles.begin(), allFiles.end());
```

# Extracting the algorithm manually

```
struct Graph {  
    Graph(int numVertices);  
    void addEdge(int from, int to);  
    vector<int> calculateLevels();  
};
```

- Creates graph explicitly
  - We can test C++ dependency discovery separately
  - We need a bit more memory
  - We can create arbitrary tests without C++ code
  - We should add `map<File*, int>`, but it's a simple algorithm
- Not a “full object”, merely wrapper around functions
- No abstract nodes types

# Takeaway: Algorithms break suddenly

**Bloomberg**

Engineering

# Takeaway: Algorithms break suddenly

1. Discover bug: Minor input change messes up output

**Bloomberg**

Engineering

# Takeaway: Algorithms break suddenly

1. Discover bug: Minor input change messes up output
2. Realize the algorithm has been “broken” for years
  - Requirements changed silently
  - Accidentally worked good enough

# Takeaway: Algorithms break suddenly

1. Discover bug: Minor input change messes up output
2. Realize the algorithm has been “broken” for years
  - Requirements changed silently
  - Accidentally worked good enough
3. Extract the mathematical algorithmic core

# Takeaway: Algorithms break suddenly

1. Discover bug: Minor input change messes up output
2. Realize the algorithm has been “broken” for years
  - Requirements changed silently
  - Accidentally worked good enough
3. Extract the mathematical algorithmic core
4. Fix the mathematical definition
  - What is a “level” if there are cycles? LLM: “Treat each strongly connected component (SCC) as a single node,” “Switch to shortest-path layering,”...

# Takeaway: Algorithms break suddenly

1. Discover bug: Minor input change messes up output
2. Realize the algorithm has been “broken” for years
  - Requirements changed silently
  - Accidentally worked good enough
3. Extract the mathematical algorithmic core
4. Fix the mathematical definition
  - What is a “level” if there are cycles? LLM: “Treat each strongly connected component (SCC) as a single node,” “Switch to shortest-path layering”...
5. Create a migration plan: Do you need to “fix” old data?

**Bloomberg**

Engineering



### 3. Bugs hide until they don't

**Bloomberg**

Engineering

# Heterogenous comparator

```
// "x", 1, 3, 5, "2", "04", "6" --> 1 "2" 3 "04" 5 "6" "x"  
using Data = variant<int, string>;
```

```
bool compare(const Data &a, const Data &b) {  
  
    return a < b;  
}
```

**Bloomberg**

Engineering

# Heterogenous comparator

```
// "x", 1, 3, 5, "2", "04", "6" --> 1 "2" 3 "04" 5 "6" "x"
using Data = variant<int, string>;
optional<int> toInt(const Data &v) {

}

bool compare(const Data &a, const Data &b) {
    auto ai = toInt(a), bi = toInt(b);
    if (ai && bi) return *ai < *bi;
    return a < b;
}
```

# Heterogenous comparator

```
// "x", 1, 3, 5, "2", "04", "6" --> 1 "2" 3 "04" 5 "6" "x"
using Data = variant<int, string>;
optional<int> toInt(const Data &v) {
    try { return get<int>(v); } catch (...) {}
}

bool compare(const Data &a, const Data &b) {
    auto ai = toInt(a), bi = toInt(b);
    if (ai && bi) return *ai < *bi;
    return a < b;
}
```

# Heterogenous comparator

```
// "x", 1, 3, 5, "2", "04", "6" --> 1 "2" 3 "04" 5 "6" "x"
using Data = variant<int, string>;
optional<int> toInt(const Data &v) {
    try { return get<int>(v); } catch (...) {}
    try { return stoi(get<string>(v)); } // TODO: improve parsing
    catch (...) { return nullopt; }
}
bool compare(const Data &a, const Data &b) {
    auto ai = toInt(a), bi = toInt(b);
    if (ai && bi) return *ai < *bi;
    return a < b;
}
```

# Testing via std::sort

```
// 1 2 "3" 4 "x"
```

```
sort(..., compare);
```

**Bloomberg**

Engineering

# Testing via std::sort

```
// 1 2 "3" 4 "x"
```

```
sort(..., compare); // msvc, gcc - ok
```

**Bloomberg**

Engineering

# Testing via std::sort

```
// 1 2 "3" 4 "x"
```

```
sort(..., compare); // msvc, gcc - ok
```

```
stable_sort(..., compare); // msvc, gcc - ok
```

**Bloomberg**

Engineering

# Testing via std::sort

```
// 1 2 "3" 4 "x"
```

```
sort(..., compare); // msvc, gcc - ok
```

```
stable_sort(..., compare); // msvc, gcc - ok
```

```
// 1 2 "3" 4 "$"
```

```
sort(..., compare);
```

# Testing via std::sort

```
// 1 2 "3" 4 "x"
```

```
sort(..., compare); // msvc, gcc - ok
```

```
stable_sort(..., compare); // msvc, gcc - ok
```

```
// 1 2 "3" 4 "$"
```

```
sort(..., compare); // msvc, gcc - ok
```

# Testing via std::sort

```
// 1 2 "3" 4 "x"
```

```
sort(..., compare); // msvc, gcc - ok
```

```
stable_sort(..., compare); // msvc, gcc - ok
```

```
// 1 2 "3" 4 "$"
```

```
sort(..., compare); // msvc, gcc - ok
```

```
stable_sort(..., compare); // msvc - ok
```

# Testing via std::sort

```
// 1 2 "3" 4 "x"
```

```
sort(..., compare); // msvc, gcc - ok
```

```
stable_sort(..., compare); // msvc, gcc - ok
```

```
// 1 2 "3" 4 "$"
```

```
sort(..., compare); // msvc, gcc - ok
```

```
stable_sort(..., compare); // msvc - ok
```

```
// gcc: 1 2 4 "$" "3"
```

Even with `-D_GLIBCXX_DEBUG` and sanitizers

**Bloomberg**

Engineering

# What went wrong?

- Undefined behaviour [alg.sorting.general]/4.1:
  - `comp(a, b) && comp(b, c)` **implies** `comp(a, c)`
  - Otherwise, it's *impossible* to define behavior

**Bloomberg**

Engineering

# What went wrong?

- Undefined behaviour `[alg.sorting.general]/4.1`:
  - `comp(a, b) && comp(b, c)` **implies** `comp(a, c)`
  - Otherwise, it's *impossible* to define behavior
- The comparator is non-transitive; there is a cycle
  - `a="3"` comes before `b=4` (as numbers)
  - `b=4` comes before `c="$"` (ints before strings)
  - `a="3"` comes *after* `c="$"` (by ASCII code)

# What went wrong?

- Undefined behaviour `[alg.sorting.general]/4.1`:
  - `comp(a, b) && comp(b, c)` **implies** `comp(a, c)`
  - Otherwise, it's *impossible* to define behavior
- The comparator is non-transitive; there is a cycle
  - `a="3"` comes before `b=4` (as numbers)
  - `b=4` comes before `c="$"` (ints before strings)
  - `a="3"` comes *after* `c="$"` (by ASCII code)
- Different implementations work differently because of UB
  - Some algorithms might freeze or crash
  - Hence, testing the comparator through sort is too black-box

# Characterization tests

- Add black-box tests before fixing
  - Mutation testing is a great metric
  - Test for [load-bearing] bugs too, do not fix
  - Mathematical model or consistency is not important first
  - That will also test engineering (e.g., overload resolution, templates)
- Aim for coverage and confidence, not DRY
- Refactor if tests and algorithm are far away
- Once you see tests: Understand when behaviour actually changes
  - Everything works if strings start with ASCII code 58+

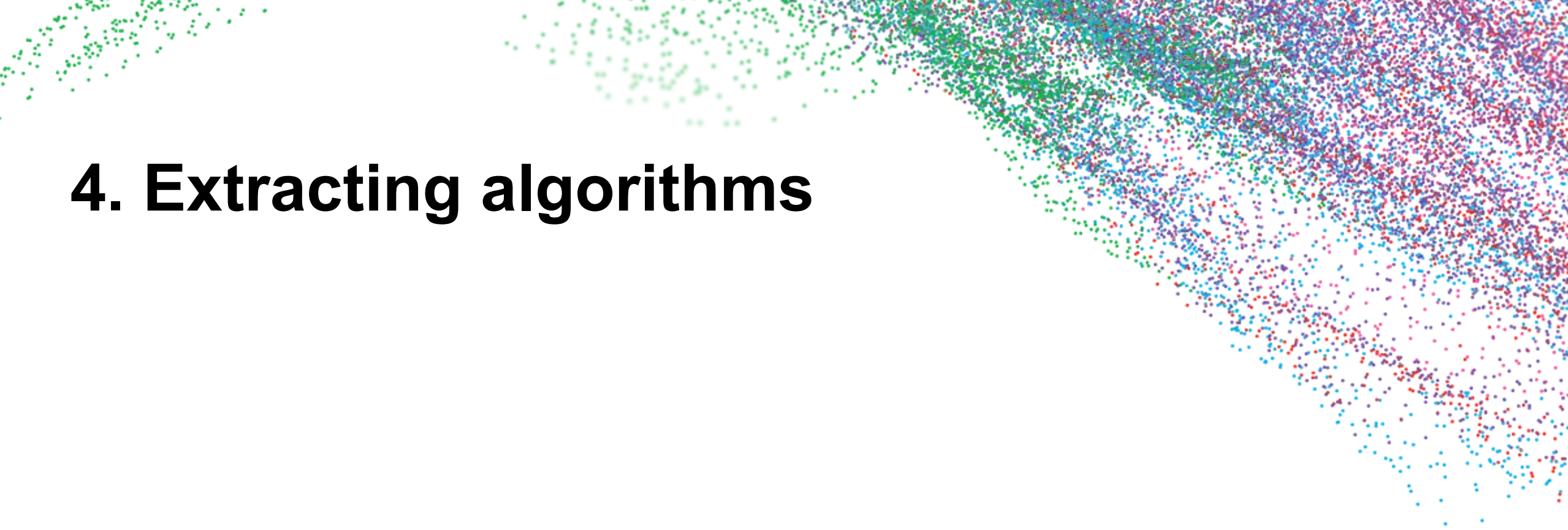
# Tests might pass accidentally

```
pair<int, int> d[]{{1, 5}, {1, 2}, {6, 6}, {2, 2}, {5, 1}};  
sort(begin(d), end(d), [](auto a, auto b) {  
    return a.first <= b.first && a.second < b.second;  
});  
// (1, 2) (1, 5) (2, 2) (5, 1) (6, 6)
```

- Comparator is non-transitive
- The output is still lexicographically sorted

# Takeaway: Algorithmic bug is a “logical UB” timebomb

- An application works for ten years without complaints
  - Old sorting algorithm did not care about buggy comparator
- New code is introduced and thoroughly tested
- The application crashes
- New code is correct, but does not tolerate an old bug



## 4. Extracting algorithms

**Bloomberg**

Engineering

# SUBCASE: Original code

```
TEST_CASE("A vector with some items") {  
    vector<int> v(5);  
    SUBCASE("resize to 1") {  
        v.resize(1);  
        CHECK(v.size() == 1);  
    }  
    SUBCASE("resize to 5") {  
        v.resize(5);  
        CHECK(v.size() == 5);  
    }  
}
```

## SUBCASE: Run 1/2

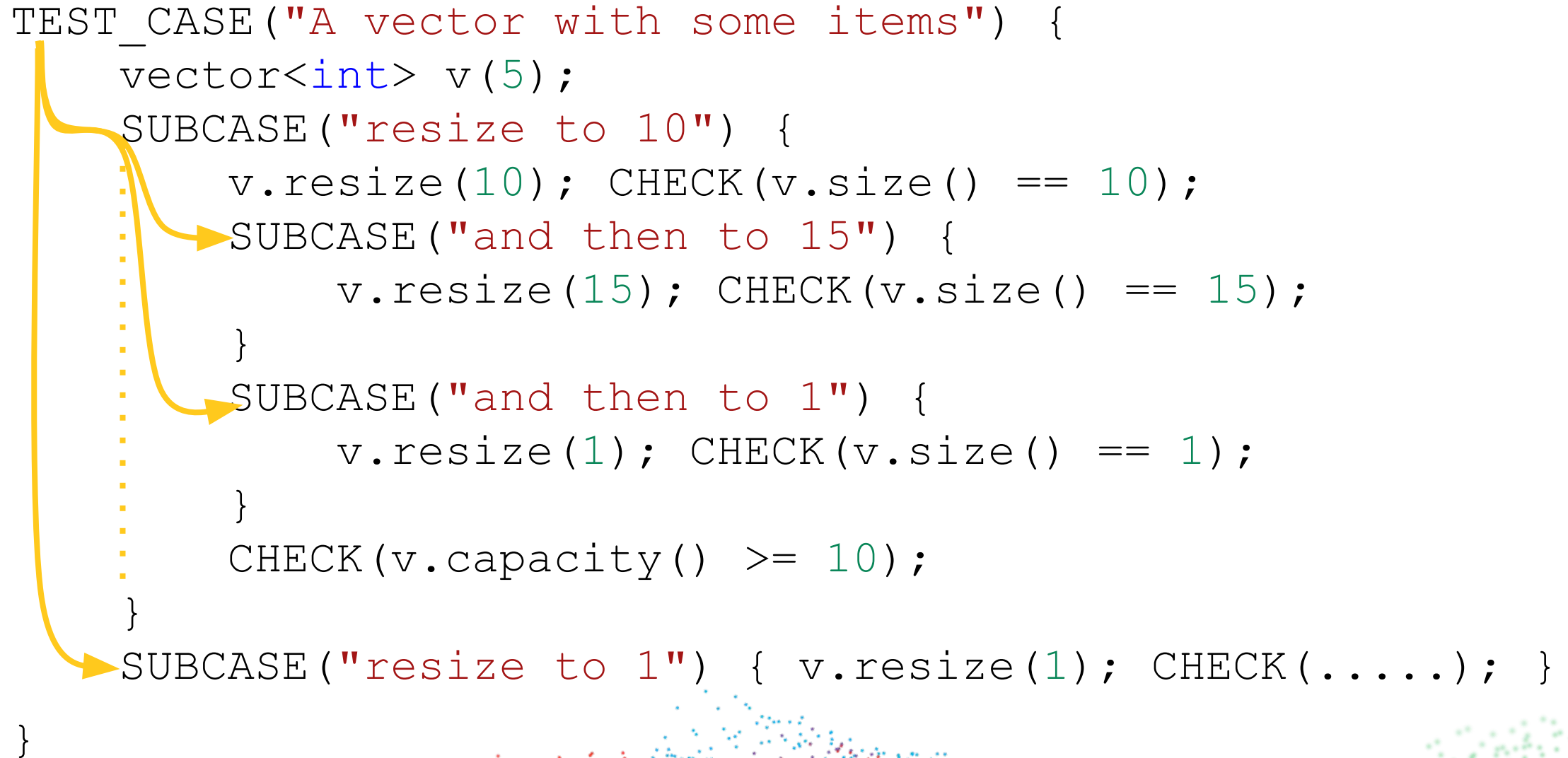
```
TEST_CASE("A vector with some items") {  
    vector<int> v(5);  
    SUBCASE("resize to 1") {  
        v.resize(1);  
        CHECK(v.size() == 1);  
    }  
    SUBCASE("resize to 5") {  
        v.resize(5);  
        CHECK(v.size() == 5);  
    }  
}
```

## SUBCASE: Run 2/2

```
TEST_CASE("A vector with some items") {  
    vector<int> v(5);  
    SUBCASE("resize to 1") {  
        v.resize(1);  
        CHECK(v.size() == 1);  
    }  
    SUBCASE("resize to 5") {  
        v.resize(5);  
        CHECK(v.size() == 5);  
    }  
}
```

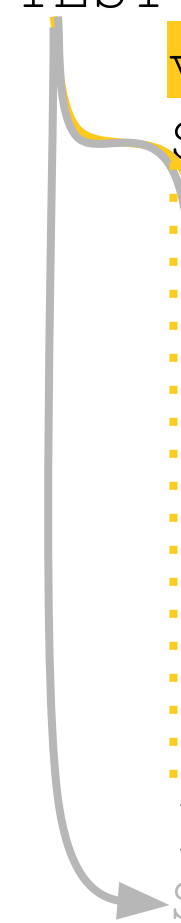
## Nested SUBCASE: Original code

```
TEST_CASE("A vector with some items") {  
    vector<int> v(5);  
    SUBCASE("resize to 10") {  
        v.resize(10); CHECK(v.size() == 10);  
        SUBCASE("and then to 15") {  
            v.resize(15); CHECK(v.size() == 15);  
        }  
        SUBCASE("and then to 1") {  
            v.resize(1); CHECK(v.size() == 1);  
        }  
        CHECK(v.capacity() >= 10);  
    }  
    SUBCASE("resize to 1") { v.resize(1); CHECK(...); }  
}
```

A diagram illustrating nested subcases. A solid yellow arrow starts at the opening curly brace of the main test case and points to the opening curly brace of the first subcase, "resize to 10". A vertical dotted yellow line extends from the opening brace of the first subcase down to the opening brace of the second subcase, "resize to 1". A solid yellow arrow points from the opening brace of the first subcase to the opening brace of the second subcase. Another solid yellow arrow points from the opening brace of the second subcase to the opening brace of the third subcase, "and then to 1". A final solid yellow arrow points from the opening brace of the main test case to the opening brace of the last subcase, "resize to 1".

## Nested SUBCASE: Run 1/3

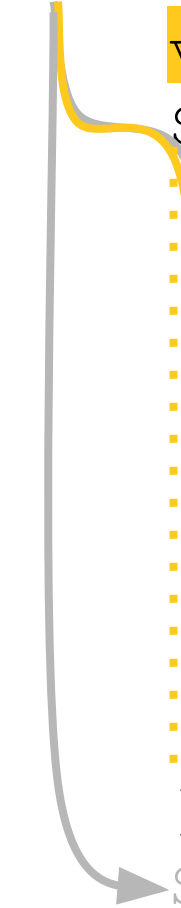
```
TEST CASE("A vector with some items") {  
    vector<int> v(5);  
    SUBCASE("resize to 10") {  
        v.resize(10); CHECK(v.size() == 10);  
        SUBCASE("and then to 15") {  
            v.resize(15); CHECK(v.size() == 15);  
        }  
        SUBCASE("and then to 1") {  
            v.resize(1); CHECK(v.size() == 1);  
        }  
        CHECK(v.capacity() >= 10);  
    }  
    SUBCASE("resize to 1") { v.resize(1); CHECK(...); }  
}
```



The diagram illustrates the execution flow of the nested subcases. A solid grey arrow starts at the beginning of the test case and follows the outermost block. A yellow arrow branches off to the first subcase, "resize to 10", and then another yellow arrow branches off to its nested subcase, "and then to 15". A grey arrow then branches off to the subcase "and then to 1". A vertical dotted line marks the end of the first subcase block, and a grey arrow continues from the bottom of this block to the next subcase, "resize to 1".

## Nested SUBCASE: Run 2/3

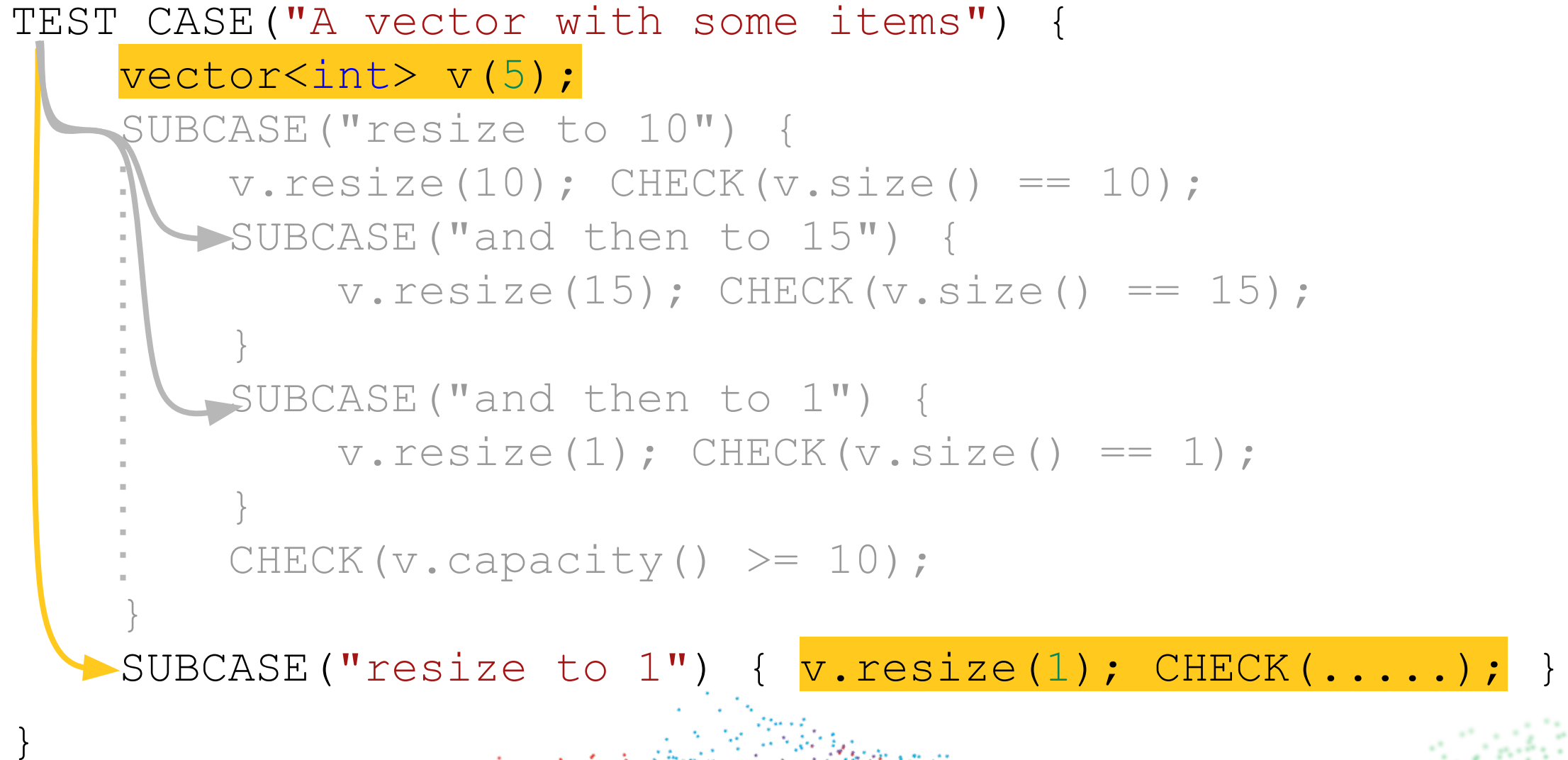
```
TEST CASE("A vector with some items") {  
    vector<int> v(5);  
    SUBCASE("resize to 10") {  
        v.resize(10); CHECK(v.size() == 10);  
        SUBCASE("and then to 15") {  
            v.resize(15); CHECK(v.size() == 15);  
        }  
        SUBCASE("and then to 1") {  
            v.resize(1); CHECK(v.size() == 1);  
        }  
        CHECK(v.capacity() >= 10);  
    }  
    SUBCASE("resize to 1") { v.resize(1); CHECK(...); }  
}
```



The diagram illustrates the execution flow of the nested subcases. A solid grey arrow starts at the beginning of the test case and points to the 'resize to 1' subcase at the bottom. A yellow arrow starts at the 'vector<int> v(5);' line and points to the 'and then to 1' subcase. A vertical dotted line is positioned to the left of the 'resize to 10' subcase block, and a grey arrow points from this line to the 'and then to 15' subcase, indicating the sequence of execution.

## Nested SUBCASE: Run 3/3

```
TEST CASE("A vector with some items") {  
    vector<int> v(5);  
    SUBCASE("resize to 10") {  
        v.resize(10); CHECK(v.size() == 10);  
        SUBCASE("and then to 15") {  
            v.resize(15); CHECK(v.size() == 15);  
        }  
        SUBCASE("and then to 1") {  
            v.resize(1); CHECK(v.size() == 1);  
        }  
        CHECK(v.capacity() >= 10);  
    }  
    SUBCASE("resize to 1") { v.resize(1); CHECK(...); }  
}
```

A diagram illustrating the execution flow of a nested subcase test. A yellow arrow starts at the beginning of the test case and points to the 'resize to 1' subcase. A grey arrow starts at the 'resize to 10' subcase and points to the 'and then to 15' subcase. A dotted line indicates the flow from the 'and then to 15' subcase to the 'and then to 1' subcase. The 'and then to 1' subcase is highlighted with a yellow background.

# Loop-generated SUBCASE

```
TEST_CASE("A vector with some items") {  
    vector<int> v(5);  
    for (int i = 5; i <= 10; ++i)  
        SUBCASE(format("resize to {}", i)) {  
            v.resize(i);  
            CHECK(v.size() == i);  
            CHECK(v.capacity() >= 10);  
        }  
}
```

- 2022: Fixed in <https://github.com/doctest/doctest/issues/597>
- 2026: Actually broke <https://github.com/doctest/doctest/issues/939>

# Why extract?

- Separate engineering and algorithmic bugs
- Possible engineering bugs: N
  - String lifetimes
  - Does not work with `std::string`
  - Logging, formatting, crash handling
  - Lack of macro escaping
- Possible algorithmic bugs: M
  - Loops indefinitely
  - Runs two SUBCASEs during a run
- Only need  $N+M$  tests, not  $N*M$

# Proposed algorithm API

```
withSubcases ([&] (Subcases &s) {  
    vector<int> v(5);  
    if (s.enterSubcase()) {  
        v.resize(10); CHECK(v.size() == 10);  
        if (s.enterSubcase()) {  
            v.resize(15); CHECK(v.size() == 15); s.exit();  
        }  
        if (s.enterSubcase()) {  
            v.resize(1); CHECK(v.size() == 1); s.exit();  
        }  
        CHECK(v.capacity() >= 10); s.exit();  
    }  
    if (s.enterSubcase()) { v.resize(1); ... s.exit(); }  
});
```



## Proposed algorithm API: Run 1/3

```
withSubcases([&](Subcases &s) {  
    vector<int> v(5);  
    if (s.enterSubcase()) {  
        v.resize(10); CHECK(v.size() == 10);  
        if (s.enterSubcase()) {  
            v.resize(15); CHECK(v.size() == 15); s.exit();  
        }  
        if (s.enterSubcase()) {  
            v.resize(1); CHECK(v.size() == 1); s.exit();  
        }  
        CHECK(v.capacity() >= 10); s.exit();  
    }  
    if (s.enterSubcase()) { v.resize(1); ... s.exit(); }  
});
```



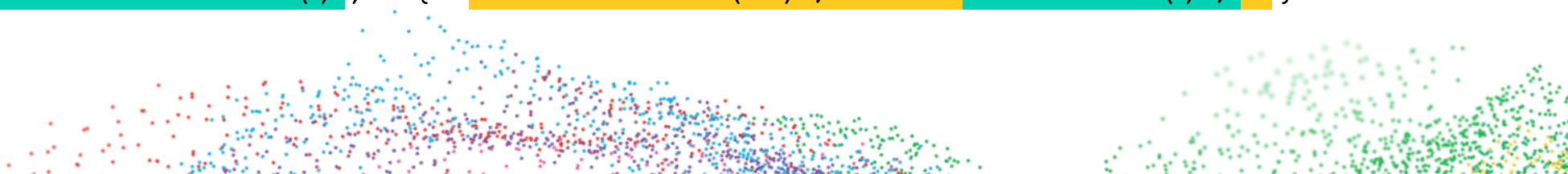
## Proposed algorithm API: Run 2/3

```
withSubcases([&](Subcases &s) {  
    vector<int> v(5);  
    if (s.enterSubcase()) {  
        v.resize(10); CHECK(v.size() == 10);  
        if (s.enterSubcase()) {  
            v.resize(15); CHECK(v.size() == 15); s.exit();  
        }  
        if (s.enterSubcase()) {  
            v.resize(1); CHECK(v.size() == 1); s.exit();  
        }  
        CHECK(v.capacity() >= 10); s.exit();  
    }  
    if (s.enterSubcase()) { v.resize(1); ... s.exit(); }  
});
```



## Proposed algorithm API: Run 3/3

```
withSubcases([&](Subcases &s) {  
    vector<int> v(5);  
    if (s.enterSubcase()) {  
        v.resize(10); CHECK(v.size() == 10);  
        if (s.enterSubcase()) {  
            v.resize(15); CHECK(v.size() == 15); s.exit();  
        }  
        if (s.enterSubcase()) {  
            v.resize(1); CHECK(v.size() == 1); s.exit();  
        }  
        CHECK(v.capacity() >= 10);  
    }  
    if (s.enterSubcase()) { v.resize(1); ... s.exit(); }  
});
```



# Proposed mathematical model

- There is a tree of SUBCASE, execute all leafs
- Assume the tree is the same between runs
- The tree is implicit
  - We can see children only if the node is executed
  - No dry runs: If we read children, we must continue to a leaf
- Testing algorithm separately is optional
  - Black-box tests with macros are more readable
  - But corner cases are still white-box
- Took me a few full days for the first attempt

# Use more abstract, mathematical API

**TechAtBloomberg.com**

© 2026 Bloomberg Finance L.P. All rights reserved.

**Bloomberg**

Engineering

# Use more abstract, mathematical API

| Harder to reason about                          | Simpler to reason about |
|-------------------------------------------------|-------------------------|
| <code>Subcase(name, line, parentSubcase)</code> | <code>Subcase()</code>  |

# Use more abstract, mathematical API

| Harder to reason about                          | Simpler to reason about                  |
|-------------------------------------------------|------------------------------------------|
| <code>Subcase(name, line, parentSubcase)</code> | <code>Subcase()</code>                   |
| Strings to the back, unless it's a number       | Induces equality classes and total order |

# Use more abstract, mathematical API

| Harder to reason about                          | Simpler to reason about                  |
|-------------------------------------------------|------------------------------------------|
| <code>Subcase(name, line, parentSubcase)</code> | <code>Subcase()</code>                   |
| Strings to the back, unless it's a number       | Induces equality classes and total order |
| Too abstract/C++                                | Textbook Pascal                          |
| <code>addDep(Node, Node)</code>                 | <code>void addDep(int, int)</code>       |

# Use more abstract, mathematical API

| Harder to reason about                          | Simpler to reason about                  |
|-------------------------------------------------|------------------------------------------|
| <code>Subcase(name, line, parentSubcase)</code> | <code>Subcase()</code>                   |
| Strings to the back, unless it's a number       | Induces equality classes and total order |
| Too abstract/C++                                | Textbook Pascal                          |
| <code>addDep(Node, Node)</code>                 | <code>void addDep(int, int)</code>       |
| <code>random_access_iterator</code>             | <code>T getAt(int)</code>                |

# Use more abstract, mathematical API

| Harder to reason about                          | Simpler to reason about                  |
|-------------------------------------------------|------------------------------------------|
| <code>Subcase(name, line, parentSubcase)</code> | <code>Subcase()</code>                   |
| Strings to the back, unless it's a number       | Induces equality classes and total order |
| Too abstract/C++                                | Textbook Pascal                          |
| <code>addDep(Node, Node)</code>                 | <code>void addDep(int, int)</code>       |
| <code>random_access_iterator</code>             | <code>T getAt(int)</code>                |
| <code>operator&lt;</code>                       | <code>operator&lt;=&gt;</code>           |



# 5. Signs of an algorithm

# Computer Science-adjacent

- “Almost standard library”, “almost textbook”
  - “Find the shortest/easiest/cheapest/minimum...”
- Lots of theory exist
  - Graph, tree, traversals, “node”, “tree”
  - Comparison, sorting, search
  - String matching, tokenization
  - Permutations, cycles
  - Multithreading, lock-free
- “This can never happen because...”
  - Better: “the invariant is... as in ... algorithm”

# Suspicious code behaviour

- Quirky behaviour
  - Minor modifications to the input change output unexpectedly
  - “Obviously works”, but needs several debugging iterations
  - Different behaviour with a different hashtable
- Small code, but need a lot of context to debug
  - Many paths reach the same branch with wildly different state
  - You need to trace intermediate state to understand what happened
  - “What if A happened, but B didn't, except after C?”

# Suspicious code structure

- It is easier to show how a variable changes than to name it
  - flag, counter, iteration

# Suspicious code structure

- It is easier to show how a variable changes than to name it
  - flag, counter, iteration
- Algorithmic code structure
  - Recursion, backtracking
  - Stack, queue, priority queue, heap
  - Prone to off-by-one errors (e.g., intervals intersection)

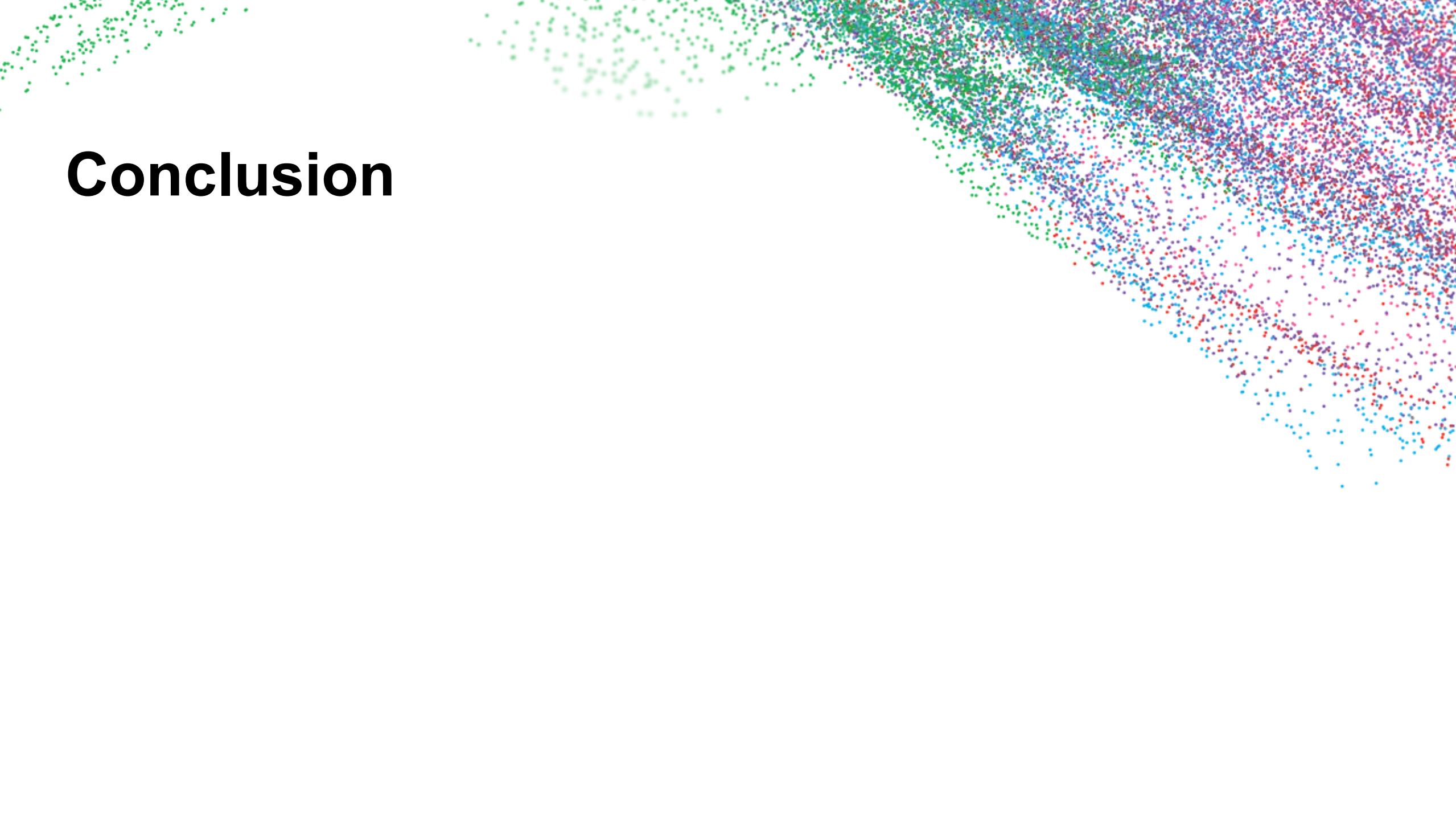
# Suspicious code structure

- It is easier to show how a variable changes than to name it
  - flag, counter, iteration
- Algorithmic code structure
  - Recursion, backtracking
  - Stack, queue, priority queue, heap
  - Prone to off-by-one errors (e.g., intervals intersection)
- “If you change this code, change code at F.cpp too”

# Suspicious code structure

- It is easier to show how a variable changes than to name it
  - flag, counter, iteration
- Algorithmic code structure
  - Recursion, backtracking
  - Stack, queue, priority queue, heap
  - Prone to off-by-one errors (e.g., intervals intersection)
- “If you change this code, change code at F.cpp too”
- Hidden state that does not map to input/output
  - "Already processed", "visited", memoization
  - Tests are stateful and corner cases require special set-up

# Conclusion



# Takeaways

- It might be an algorithm even if it's not from a textbook

# Takeaways

- It might be an algorithm even if it's not from a textbook
- Usual tests are not enough for algorithmic bugs

# Takeaways

- It might be an algorithm even if it's not from a textbook
- Usual tests are not enough for algorithmic bugs
- Each algorithm has a name, math model, and corner cases

# Takeaways

- It might be an algorithm even if it's not from a textbook
- Usual tests are not enough for algorithmic bugs
- Each algorithm has a name, math model, and corner cases
- An algorithm might break long after assumptions changed

# Takeaways

- It might be an algorithm even if it's not from a textbook
- Usual tests are not enough for algorithmic bugs
- Each algorithm has a name, math model, and corner cases
- An algorithm might break long after assumptions changed
- Incorrect algorithm might only break after its users change

# Takeaways

- It might be an algorithm even if it's not from a textbook
- Usual tests are not enough for algorithmic bugs
- Each algorithm has a name, math model, and corner cases
- An algorithm might break long after assumptions changed
- Incorrect algorithm might only break after its users change
- Refactor algorithmic core into a separate component

# Bloomberg

Engineering

# Thank you!

**TechAtBloomberg.com**

© 2026 Bloomberg Finance L.P. All rights reserved.

# Questions?

Interested in C++ @ Bloomberg?

We are hiring: [bloomberg.com/engineering](https://bloomberg.com/engineering)

Career Opportunities



Learn more about our work



**TechAtBloomberg.com**

**Bloomberg**  
**Engineering**

# Bonus: What tests to add

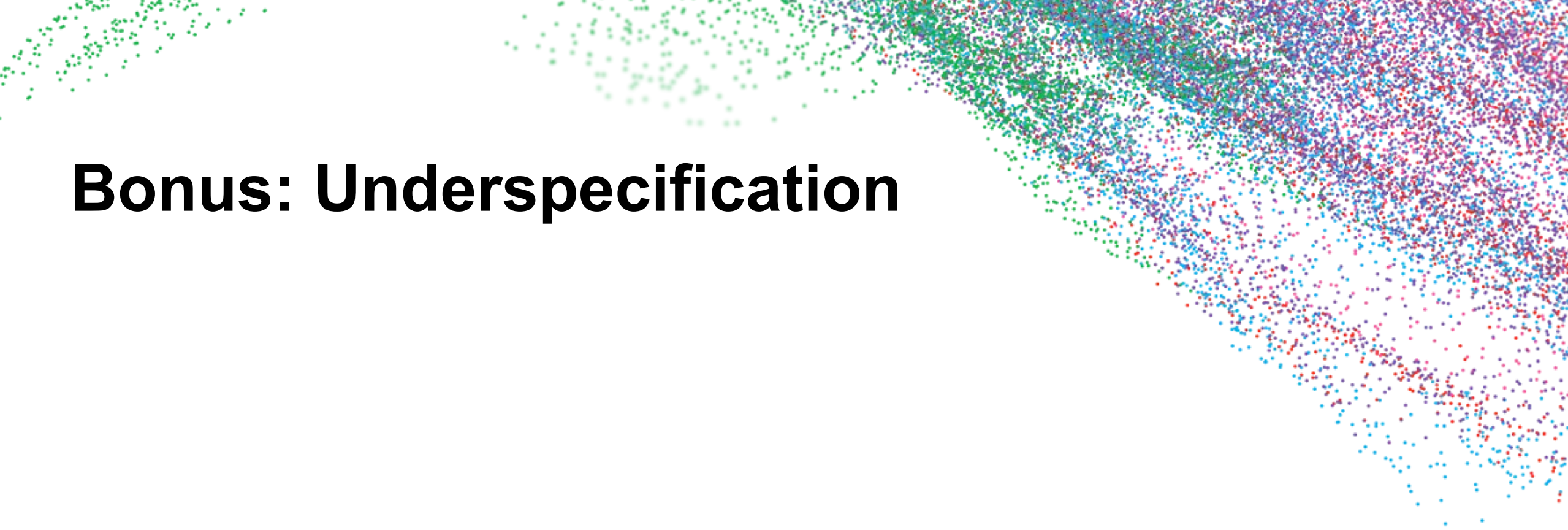
- Smoke test
- Minimum size, maximum size: input, output, intermediates
- Beautiful tests, even if no other reason
- All possible small inputs
- Big random tests
- Look for symmetries in the problem: e.g., odd vs. even input size
- Cover each input/output/intermediate boundary
- Counter-test for greedy strategies
- Ugly tests: All of the above, shuffled indices-names
- Engineering boundaries: INT\_MAX, -1 / INT\_MIN...

# Bonus: Greedy algorithms

- The following are rarely *mathematically* best:
  - “The best answer so far”, “the maximum subsequence so far”
  - “It’s obviously the best next step” (but is it the best long-term?)
  - Sort, then process in order
- Example problem: How to get [0,1,4,2,6,3,5] with few moves?
  - [0,1,2,3,4,5,6] -> move 4 -> move 6

```
int last = -1;
for (int value = 0; value < n; ++value)
    if (positionOf[value] >= last) { last = positionOf[value]; }
    else { moved.emplace_back(positionValue[value]); }
```

- Incorrect: [1,2,3,4,5,0] results in five moves, not one.
- It’s the “Longest Increasing Subsequence” problem
- Full solution is  $O(n \log n)$ , but do you need it?



# Bonus: Underspecification

**TechAtBloomberg.com**

© 2026 Bloomberg Finance L.P. All rights reserved.

**Bloomberg**

**Engineering**

# Non-functional requirements

- Number of callbacks, allocations, CPU operations, memory
- How to measure?
  - Asymptotic complexity
  - Amortized or worst cast?
- “Linear” of what? What is “size of input”?

```
while (sscanf(buf, "%d%n", &val, &read) == 1) {  
    vals.push_back(val);  
    buf += read;  
}
```

# Output parameters

- Append or overwrite?

- `void loadInto(vector<int>*)`

- C strings: Is the buffer change past NULL?

- What happens with output parameters on errors?

```
const char *FMT = "xyz:%123456789123s";
```

```
_snprintf_s(buf, 4, _TRUNCATE, FMT, "x"); // xyz\0
```

```
_snprintf_s(buf, 5, _TRUNCATE, FMT, "x"); // \0yz:\0
```

- Most users do not care. Do you?

# Self-sub-object assignment

```
struct Foo {  
    string s = "hello world very long string";  
    vector<Foo> children;  
};  
  
int main() {  
    vector<Foo> f(4);  
    f[0].children.resize(2);  
    f = f[0].children; // (1)  
    // auto tmp = std::move(f[0].children); // (2)  
    // f = std::move(tmp); //  
}
```

# Sometimes precise contract is impossible

- We might only care about “existing inputs”
  - CSV parsing: No exact specification, escaping varies.
  - Integer overflows: Do not happen for  $10^2$ , happen for  $10^9$
- External dependencies might give no guarantees
  - Filesystem: Inter-thread/process visibility, time-of-check-time-of-use
  - P1 creates F.txt, then G.txt. If P2 sees F.txt, does it see G.txt?
- “Proper” solution might be too complicated/costly
  - Default SQL isolation level is not “Serializable”
  - Printing floating-point numbers: Errol (2016), Grisu3 (2010)...